CSC-239 *Java Programming*

Chapter 14: JavaFX Basics

This lecture is being recorded.

October 29, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Reminder: Project 2

- The due date for Project 2:

  Monday, November 17

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

- BHCC Tutoring:

  https://bhcc.edu/tutoring/

JavaFX vs Swing and AWT

- Historical Progression of Java GUI libraries:
  - *Abstract Windows Toolkit (AWT)*: Original Java graphical user interface (GUI).
  - *Swing components:* Swing components depend less on the target platform and use less of the native GUI resource.
  - *JavaFX* : A completely new GUI platform, intended to replace Swing.
- At some point, Oracle decided to let JavaFX be managed as an "Open Source" project.

JavaFX Online Documentation

- **OpenJFX** Documentation:

  https://openjfx.io/

Basic Structure of JavaFX

- A JavaFX application is a sub-class of the **`javafx.application.Application`** class.
- The JavaFX application overrides the `start(Stage primaryStage)` method.
- Objects in the JavaFX application:
  - Stage
  - Scene
  - Nodes

Overly Simple Example

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;
public class MyJavaFX extends Application {
   @Override
   public void start(Stage primaryStage) {
       Button btOK = new Button("OK");
       Scene scene = new Scene(btOK, 200, 250);
       primaryStage.setTitle("MyJavaFX");
       primaryStage.setScene(scene);
       primaryStage.show();
   }
   /* Usually do not need a "main" method. */
   public static void main(String[] args) {
      launch(args);
   }
}
```

Something is Missing

- This first example, MyJavaFX, is useful in illustrating the very basic program structure, and also why we need something more.
- Specifically, the "OK" button in this program occupies the entire window!

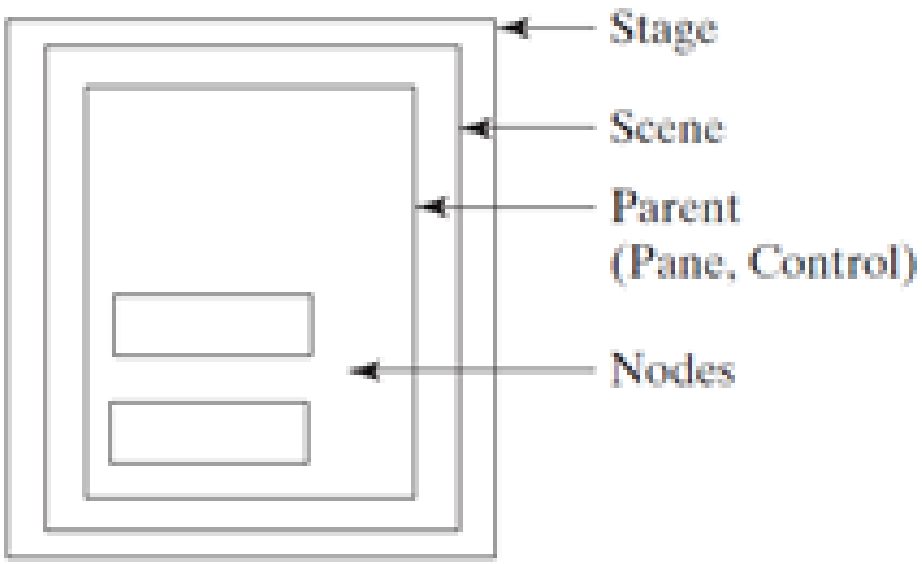


Layout Control

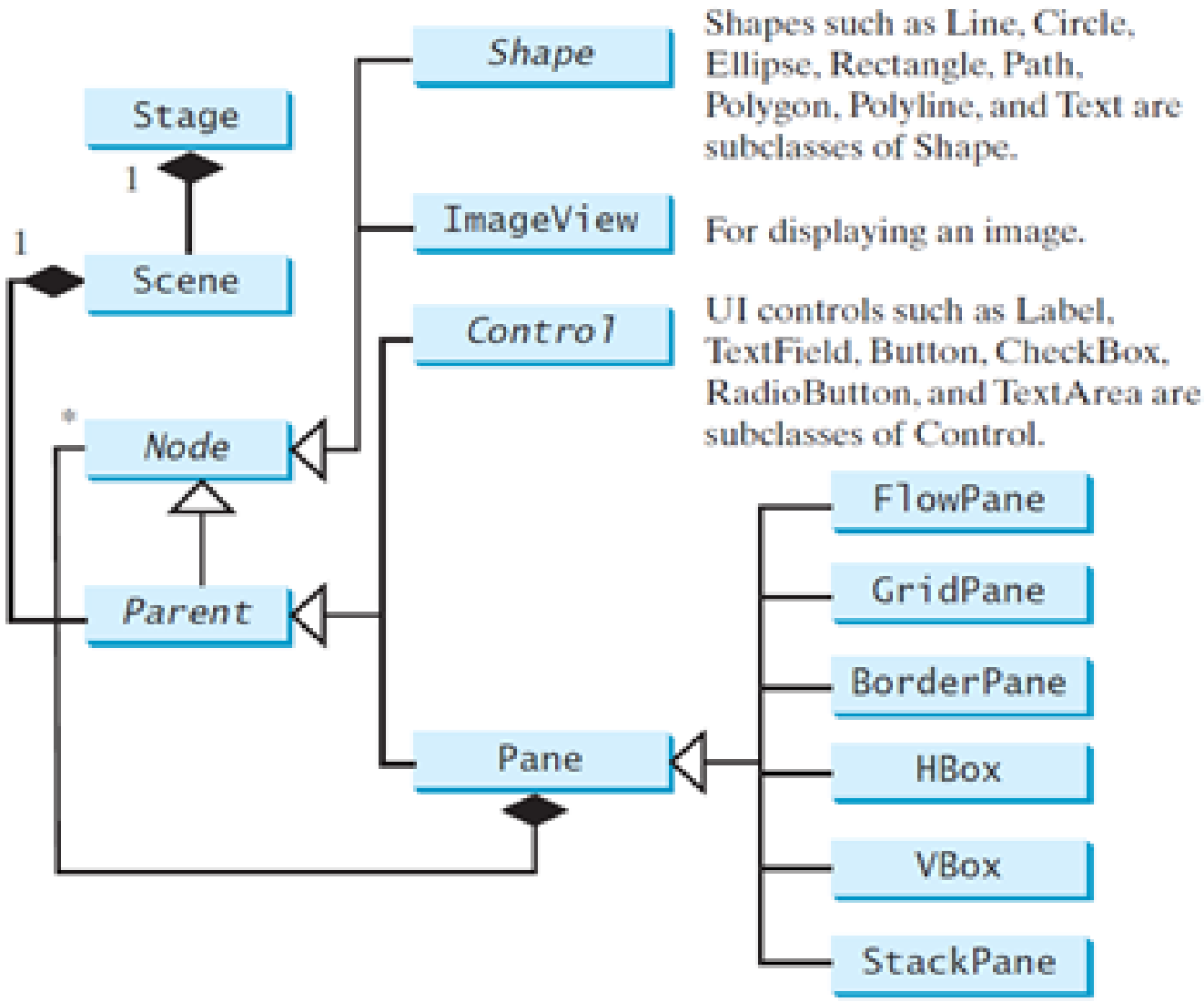
- Container classes, called **Pane**s, position the visual components (called **Node**s) in the display according to some set of rules.
- If we use a **Pane** object directly, we need to use absolute X/Y coordinates to position the **Node** objects.
- We frequently choose among several specialized sub-classes of the **Pane** class:
  - StackPane: all nodes are in the center of the pane
  - FlowPane: dynamically arranged in rows, columns
  - GridPane: two-dimensional grid
  - BorderPane: top, right, bottom, left, center regions
  - HBox: single horizontal row
  - VBox: single vertical column.

Layout of Nodes in the Display

- **Pane**: a container class for automatically laying out the nodes.
  - You place nodes inside a pane, and then place the pane into a scene.



Class Diagram (Figure 14.3, page 545)



Stack Pane Example: ButtonInPane

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;
public class ButtonInPane extends Application {
   @Override
   public void start(Stage primaryStage) {
      StackPane pane = new StackPane();
      pane.getChildren().add(new Button("OK"));
      Scene scene = new Scene(pane, 200, 50);
      primaryStage.setTitle("Button in a pane");
      primaryStage.setScene(scene);
      primaryStage.show();
   }
}
```

Order of Processing

- The JVM creates the **`primaryStage`** object, and passes it to the **`start`** method.
- The **start** method:
  - Creates a **Pane** object and adds nodes to it with

    `pane.getChildren().add(...)`
  - Creates a **Scene** object, using the **Pane**.
  - Passes the **Scene** object to the **`primaryStage`** object.

ObservableList object

- It is worth pointing out that the **`Pane.getChildren()`** method returns a reference to an object of the

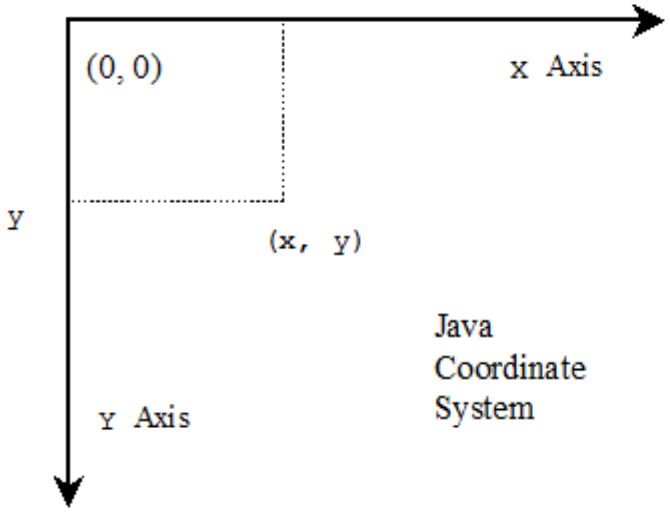
  `javafx.collections.ObservableList`

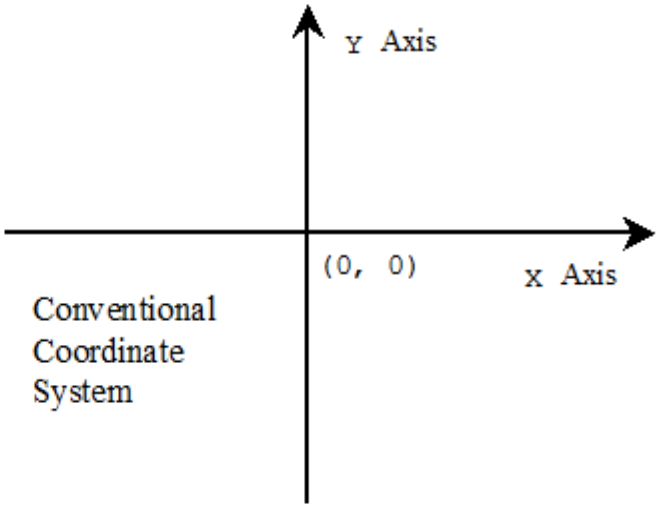
  class.
- **ObservableList** behaves like an **ArrayList** for storing a collection of elements.

  The **ArrayList** Class was introduced in section 11.11 (page 434) of the textbook. (We discussed the ArrayList class while we were covering chapter 12.)
- Calling **`pane.getChildren().add(...)`** adds an element to the list.

The Java Coordinate System

- Different from what you may remember from math courses:
  - The origin (`0,0`) is at the upper left of the display.





Absolute Coordinates: ShowCircle

```
public void start(Stage primaryStage) {
    Circle circle = new Circle();
    circle.setCenterX(100);
    circle.setCenterY(100);
    circle.setRadius(50);
    circle.setStroke(Color.BLACK);
    circle.setFill(Color.CYAN);

    Pane pane = new Pane();
    pane.getChildren().add(circle);

    Scene scene = new Scene(pane, 200, 200);
    primaryStage.setTitle("ShowCircle");
    primaryStage.setScene(scene);
    primaryStage.show();
}
```

The Color Class

- Component Colors:
  - red
  - green
  - Blue

  Each value is a double in the range of 0.0 to 1.0.
- Opacity:

  Represented by a double in the range of 0.0 to 1.0.
- Example -- white, transparent:

```
circle.setFill(new Color(1,1,1,0);
```

- Example -- white, opaque:

```
circle.setFill(new Color(1,1,1,1);
```

Binding Properties

- One object can be "bound to" another object
- If a *binding property* of the "*source*" object changes, then the binding property of the "*target_object*" object also changes.
- The target object listens to the changes in the source object and automatically updates itself.
- The target object "binds with" the source object by calling the **`bind`** method:

target_object`.bind(`*source*`);`

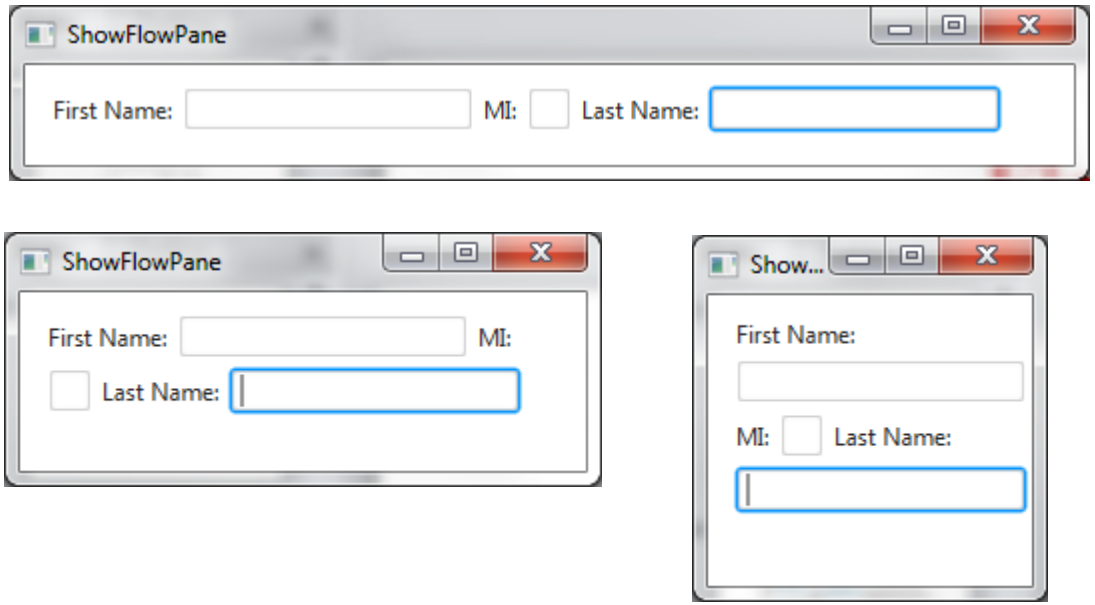
ShowCircleCentered.java

- The circle displayed in this application will be centered in the window, even *after* re-sizing, because it has the following:

```
circle.centerXProperty().
    bind(pane.widthProperty().
        divide(2));
circle.centerYProperty().
    bind(pane.heightProperty().
        divide(2));
```

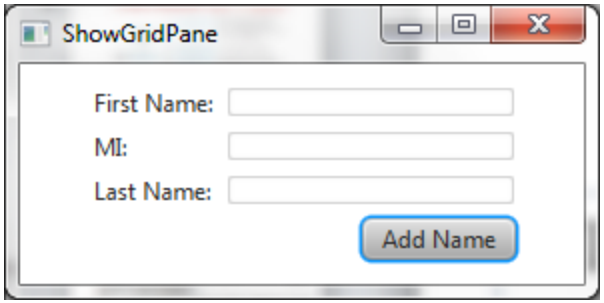
FlowPane

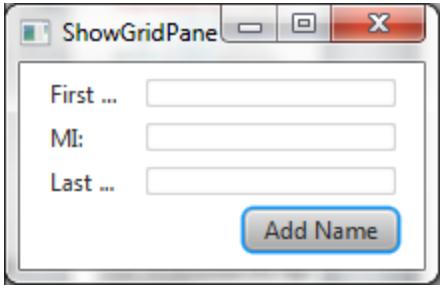
- Nodes are displayed in rows.
- Row length is automatically adjusted so all nodes can be displayed



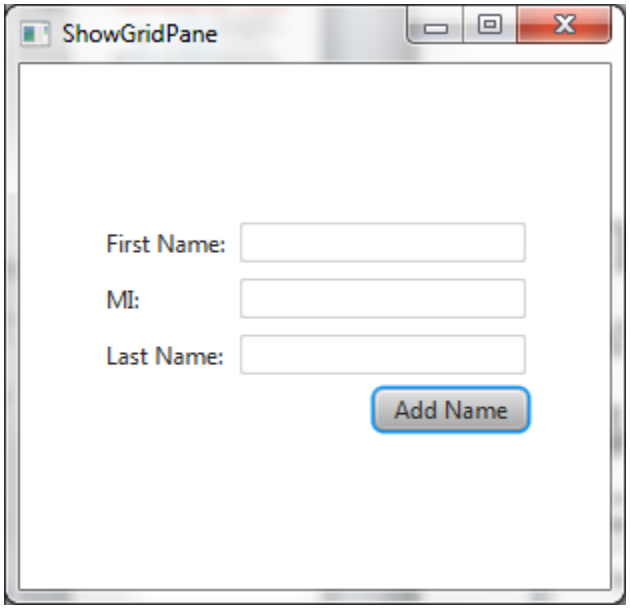
GridPane

- The nodes are placed in the specified row and column of the grid.
- Not every cell in the grid needs to be filled.





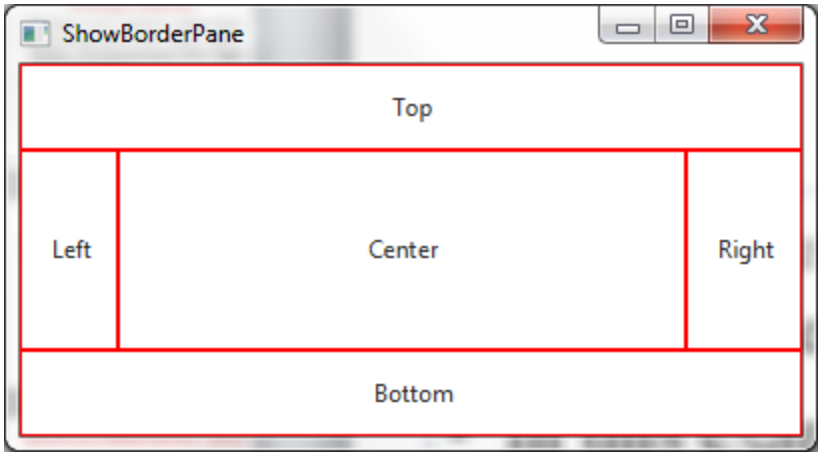


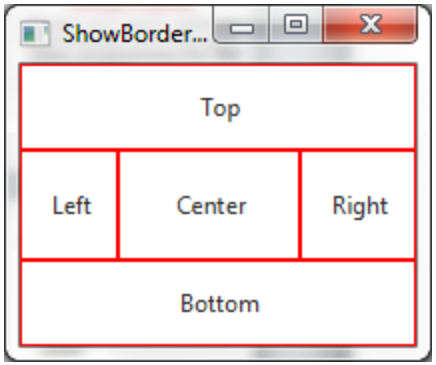


BorderPane

- The regions are “Top”, “Right”, “Bottom”, “Left”, “Center”
- In this example, each region contains a CustomPane object (subclass of StackPane).

  The CustomPane class includes a Label for each pane, as well as a red border.

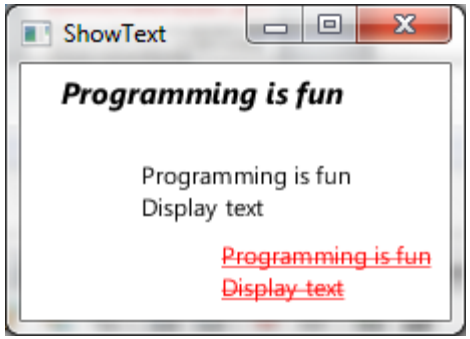




Text Nodes

```
Double x = 60, y = 60;
String text = "Programming is fun\n" +
              "Display Text";
Text text2 = new Text(x, y, text);
```





Rectangle

- Need to import Rectangle class:

```
import javafx.scene.shape.Rectangle;
```

- Rectangle position is the X-, Y- values of the upper-left corner of the rectangle:

```
double x = 10;
double y = 60;
double width = 100;
double length = 80;
Rectangle r1 =
    new Rectangle (x, y, width, height);
```

Line

- Need to import Line class:

```
import javafx.scene.shape.Line;
```

- Specify start and end X-, Y- values:

```
double startX = 10;
double startY = 60;
double endX = 100;
double endY = 80;
Line line1 =
    new Line(startX, startY, endX, endY);
```